DBMS UE24CS351A · EXPERIENTIAL LEARNING LEVEL 3

CivicFix

Municipal Civic Grievance Redressal System with Semantic Duplicate Detection and SLA-Based Auto-Escalation

Project status audit & end-to-end build plan

Team	Vismay HS (PES1UG24AM325) · Ashvin Ronald Lobo (PES1UG24AM345)
Repository	`github.com/VismayHS/CivicFix`
Prepared	25 October 2026
Scope	Runs locally end to end (Docker + Node). Cloud deployment is a later phase (Section 17).
Sources reviewed	Rubric (*DBMS Level 3 (1).pdf*), Problem Statement Proposal, EL3 Schema & DDL PDF, Excalidraw ER file, Lab 4 Google Doc (18 pages, all screenshots), the CivicFix repo, and the 7 toolkit repos you listed

Contents

1 Summary
2 Rubric scorecard: what's done, what's left
3 What exists today (evidence)
4 Two schemas disagree: the mismatch list
5 Technology decision & justification
6 Local architecture
7 Final schema v2: every change and why
8 Triggers, functions, procedures, views
9 Query catalogue (DML, nested, correlated, recursive, joins)
10 Vector database design (Qdrant)
11 Authentication & RBAC in the database
12 Next.js application: pages, API, UX
13 Toolkit: where each repo is used
14 Repository layout & local setup
15 Phased build plan with owners
16 Testing, demo script & viva prep
17 Future: deployment plan
18 Resume & portfolio
19 Risks & mitigations

1 Summary

~6 / 50
marks you have evidence for today (ER, relational schema, part of the DDL), and only after those artefacts are corrected

44 marks open
Vector DB (11), advanced SQL (6), queries + DML (8), app + API (5), auth/RBAC (4), docs/viva (6), and the rest

8 phases
to a working local build: setup → design fix → DB core → queries → app + auth → vector → UIs → polish/docs

The five decisions that matter

1. **Keep one schema.** Your Lab 4 database and your EL3 Schema PDF describe *different* databases (Section 4). Evolve the Lab 4 schema, which actually runs in Postgres and has the Chen ER diagram, into a single **schema v2** that takes the good parts of the EL3 PDF. Then redraw the ER diagram and the relational schema so both match the DDL exactly. The rubric grades that consistency.
2. **PostgreSQL 17 + Qdrant** (Option 2: SQL + Vector, as your proposal committed). Qdrant is a separate, purpose-built vector database. That makes "NoSQL/Vector database creation (4)" and "Vector feature RUD (7)" visible and easy to defend, worth 11 marks together. pgvector is the documented alternative.
3. **Make the database do the work.** SLA deadlines, auto-assignment, status-machine validation, history, notifications and escalation all live in Postgres as triggers, functions and procedures, scheduled by `pg_cron`. That is exactly what Advanced Database Concepts (6) rewards, and it makes for a strong viva answer.
4. **Enforce RBAC inside Postgres, not only in the app:** database roles, `GRANT`s and Row-Level Security policies. The rubric says "role-based access control *in database*". Most teams only check roles in their app code.
5. **Build a "DB Lab" and a "Vector Console" into the admin UI.** Every rubric query, procedure and vector operation becomes runnable live, with its SQL and results shown. Evaluators see every rubric item in two clicks.

> **Biggest risk: originality & viva (4 marks)**
> Evaluators will ask each of you to modify code live. If AI writes most of the project, you both still have to understand every line you present. The proposal splits the work as Ashvin = database layer, Vismay = Next.js app, auth, API and vector integration. Each of you should author and commit your own part from your own GitHub account so the history shows individual contribution. Check your course's AI-use policy and be open about it. Commits made through Claude Code carry a `Co-Authored-By` line.

2 Rubric scorecard

Each row is one line of the official rubric. "Partial credit" is what the rubric awards for partially correct work. **Status** reflects the evidence that exists today.

#	Component	Max	Partial	Status	What full marks needs (and where in this plan)
1	ER diagram	2	1	PARTIAL	A Chen ER diagram exists in Lab 4 but disagrees with the DDL: weak entities, the Location attribute, Category, duplicate matches, Notification→Complaint. Redraw it for schema v2. §7
2	Relational schema with keys, relationships, constraints	3	2	PARTIAL	Two versions exist (Lab 4 diagram and EL3 PDF) and both disagree with the live DB. Publish one v2 schema diagram with PK, FK, UNIQUE and CHECK marked. §4, §7
3	SQL DDL with constraints	4	3	PARTIAL	8 tables, CHECK/FK/UNIQUE, ALTER/DROP and indexes exist, but only as psql screenshots. Commit numbered migration files for v2 to the repo. §7, §14
4	NoSQL/Vector database creation	4	3	NOT STARTED	Qdrant collection `complaints` (384-d, cosine, HNSW) with payload indexes, created by a versioned setup script. §10
5	Technology justification	1	0	PARTIAL	The proposal names the tools. A written justification with alternatives and trade-offs is needed. The text in §5 is ready to drop into the docs.
6	DML operations	4	3	NOT STARTED	Realistic seed data, plus INSERT / UPDATE / DELETE / SELECT that the app actually uses, including UPSERT, RETURNING and DELETE... USING. §9
7	Nested / correlated / recursive queries, joins	4	3	PARTIAL	The Lab 4 views have joins and one scalar subquery. Add the named catalogue: correlated, recursive CTE (duplicate chain, supervisor chain), anti-join, self-join, window functions. §9
8	Procedure/function, trigger, CTE	6	3	NOT STARTED	7 triggers, 8 functions/procedures, CTE analytics, transactions with `SKIP LOCKED`, scheduled by pg_cron. §8
9	NoSQL/Vector feature RUD	7	4	NOT STARTED	Read (similarity search with filters, retrieve, recommend), Update (payload sync, re-embed), Delete (withdrawn complaints), all wired to real features and shown in the Vector Console. §10
10	Next.js interface	3	2	NOT STARTED	Next.js 16 App Router. Public, citizen, staff and admin surfaces. §12
11	Backend/API + DB integration	2	1	NOT STARTED	`/api/v1` route handlers plus Server Actions, raw parameterized SQL, transactions per request. §12
12	Authentication + ≥2 roles + RBAC in DB	4	3	NOT STARTED	Only a `Role` column exists. Needs pgcrypto password hashing, DB-backed sessions, Postgres roles and RLS policies, and role-aware UI. §11

#	Component	Max	Partial	Status	What full marks needs (and where in this plan)
13	Documentation, Git/GitHub	2	1	STARTED	The repo exists with a 2-line README (heading typo "CivicFix-"). Needs a full README, a docs folder, issues/PRs and a clean commit history. §14, §16
14	Originality, contribution, modification/viva	4	2	PREP NEEDED	A split commit history, CONTRIBUTIONS.md, and rehearsed modification drills. §16
	Total	**50**	31		

How to read "Partial"

The rubric's partial column adds up to 31. Getting every item merely partly right caps you near 31/50. The gap between 31 and 50 comes from items being complete, correct and demonstrable. That's why this plan puts consistency between the diagrams and the DDL, plus live demonstrability, ahead of extra features.

3 What exists today

Artefact	What it contains	Verdict
DBMS Level 3 (1).pdf	Rubric only: the 7 criteria and the 14-line evaluation table (both images). Requires **Next.js**, a **NoSQL/Vector DB**, and **RBAC in the database**.	REFERENCE
Problem Statement Proposal	Domain, Option 2 (PostgreSQL + pgvector/Qdrant), 7 advanced features, 3 roles (Citizen, Staff, Ward/Municipal Admin), a state machine Open→Assigned→In Progress→Resolved→Closed/Reopened, and the team split.	GOOD BASIS
DBMS_EL3_Schema_and_DDL.pdf	11 tables (plural names, `ComplaintCategories` with `sla_hours`, `DuplicateMatches`, `VECTOR(384)` column), indexes incl. GIN full-text and ivfflat, one view, illustrative ALTER/DROP. **Never executed**, as far as the evidence shows.	DESIGN ONLY
Grievance_System_ER_Diagram.excalidraw	11 entity boxes with PK/FK and 1:N lines. It mirrors the EL3 PDF exactly. It is a *logical/relational* diagram, not a Chen ER diagram.	LOGICAL DIAGRAM
Lab 4 doc (Google Doc → DBMS_Lab4.pdf)	The Chen ER diagram (USER, COMPLAINT, WARD, DEPARTMENT, NOTIFICATION; weak STATUS_HISTORY and ESCALATION; multivalued Phone_Number; derived Resolution_Time; composite Location; Duplicate Of self-relationship). A hand-drawn relational schema. psql screenshots of **8 tables actually created** (department, ward, "User", complaint, status_history, escalation, upvote, notification), ALTERs (add/drop column, add CHECK, rename, type change, set default), 13 indexes incl. a composite index experiment, and 4 views (`vw_sla_breach_candidates`, `vw_complaint_full_detail`, `vw_escalation_chain`, `vw_department_workload`).	REAL, RUNS
github.com/VismayHS/CivicFix	README.md only (150 bytes, heading reads "CivicFix-").	EMPTY

> **The DDL lives only in screenshots**
>
> The Lab 4 tables were typed into psql on an Ubuntu machine. That database isn't on this laptop, and the SQL isn't in any file. Phase 2 turns it into versioned migration files, so the database can be rebuilt with one command.

4 Two schemas disagree

An evaluator who opens your ER diagram, relational schema and DDL side by side will find these differences. Every row is resolved in schema v2 (Section 7).

Area	Lab 4 (live DB + Chen ER)	EL3 Schema PDF / Excalidraw	v2 resolution
Table names	Singular; `"User"` needs quoting (reserved word)	Plural (`Users` , `Complaints`)	Singular, snake_case; `"User"` → `app_user` (ALTER ... RENAME)
Department	`contact_email` + CHECK	`description` , `created_at`	Keep both: contact_email (CHECK) + description
Ward	`zone` , `ward_population`	`ward_number` UNIQUE, zone	ward_number UNIQUE + zone + ward_population
Phone	Multivalued in ER; single nullable column in DDL	Single, NOT NULL UNIQUE	Map the multivalued attribute properly → `user_phone` table
Staff/admin scope	User has department_id only	department_id + ward_id + role CHECK	department_id, ward_id, supervisor_id; role-consistency CHECK
Category & SLA	`category VARCHAR(50)` on complaint	`ComplaintCategories(sla_hours, department_id)`	`complaint_category` table (removes a transitive dependency, 3NF)
Status values	OPEN, IN_PROGRESS, ESCALATED, RESOLVED, CLOSED, DUPLICATE	Open, Assigned, In Progress, Resolved, Closed, Reopened	Union of both: OPEN, ASSIGNED, IN_PROGRESS, ESCALATED, RESOLVED, CLOSED, REOPENED, DUPLICATE; transitions in a table
Complaint columns	filed_by, assigned_to, resolution_time, duplicate_of, duplicate_score	citizen_id, assigned_staff_id, address, resolved_at, description_embedding	filed_by, assigned_to, address, resolved_at, duplicate_of; the embedding moves to Qdrant
Derived attribute	Resolution_Time is *derived* in ER but stored	resolved_at stored	Store `resolved_at` ; derive the duration in views (matches ER notation)
Weak entities	STATUS_HISTORY, ESCALATION drawn weak (partial keys), but DDL uses a lone SERIAL PK	Strong entities	Composite PK (complaint_id, history_no) / (complaint_id, escalation_level); ON DELETE CASCADE

Area	Lab 4 (live DB + Chen ER)	EL3 Schema PDF / Excalidraw	v2 resolution
Location	Composite attribute in ER; a *Location table* in the relational diagram that was never created	lat/long columns	Flatten the composite into latitude, longitude, address; drop the Location table from the diagram
Notification	department_id FK; no complaint link, no type	complaint_id FK + type CHECK	user_id, complaint_id, type CHECK, read_status, sent_at
Duplicates	duplicate_of self-FK + score	DuplicateMatches table + review_status	Both: `duplicate_match` records every candidate and review; duplicate_of marks the outcome
Escalation	escalated_to, level 1–5, reason	+ escalated_from, resolved_at	+ escalated_from, resolved_at; levels driven by `escalation_policy`
Diagram drift	Relational diagram says `Duplicate_ID` where the DDL says `Duplicate_Of`, and shows no Department_ID on Notification although the DDL has one. The Excalidraw file shows the EL3 PDF version, not the live one.		Regenerate both diagrams from v2

5 Technology decision & justification

This section doubles as the "Technology justification (1 mark)" text. Copy it into `docs/technology-justification.md`.

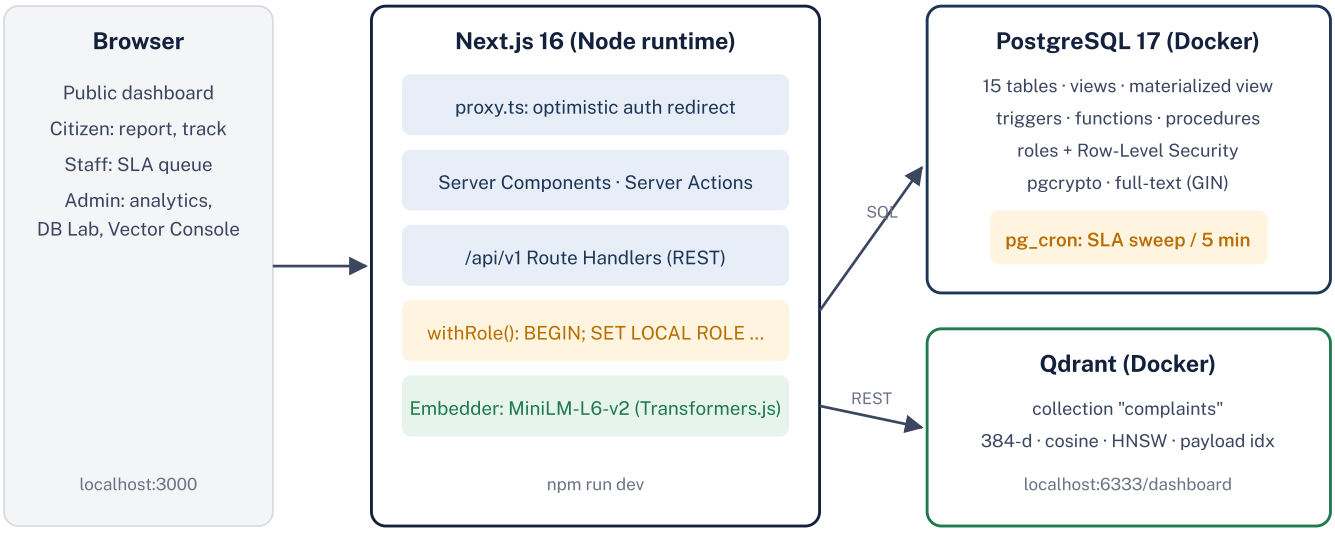
Layer	Choice	Why	Rejected alternatives
Relational DB	**PostgreSQL 17** (Docker)	The data is strongly relational: complaints link to citizens, staff, wards, departments, categories, history and escalations. Postgres gives ACID transactions, rich constraints, PL/pgSQL triggers and procedures, Row-Level Security, built-in full-text search (`tsvector` + GIN), `pgcrypto` for password hashing, and `pg_cron` for in-database scheduling. Your Lab 4 work is already in Postgres.	MySQL (no RLS, weaker procedural features); MongoDB as primary (relational integrity would be hand-rolled)
Vector DB	**Qdrant** (Docker; Qdrant Cloud later)	A purpose-built vector database. HNSW index with cosine distance; **payload filtering combined with similarity search**, so a single query covers "similar text, same ward, still open, last 60 days"; a REST API and a built-in web dashboard to show evaluators; scales independently of the transactional DB. It keeps the two paradigms clearly separate, which the rubric scores as separate creation and RUD items.	pgvector (valid, and the proposal allowed it, but it lives inside the same engine); Pinecone (cloud-only); Chroma (weaker filtering and ops story)
Embeddings	**all-MiniLM-L6-v2**, 384-d	A small sentence-embedding model that runs on CPU and is good at short English paraphrases, which is what complaints are. Locally it runs inside the Next.js server via Transformers.js, so no API key is needed. The same model is free on Qdrant Cloud Inference, so deployment needs no code change. 384 dimensions match your original `VECTOR(384)` design.	OpenAI embeddings (paid, external); larger models (slower, no gain for short text)
Web app	**Next.js 16** (App Router), React 19, TypeScript	Required by the rubric. Server Components read the DB directly. Server Actions and Route Handlers form the API. `proxy.ts` (renamed from middleware in v16) handles optimistic auth redirects.	
DB access	**postgres.js**, raw SQL	Parameterized tagged-template SQL, so evaluators see real SQL rather than ORM calls. It also handles per-request transactions with `SET LOCAL ROLE` for RLS.	Prisma/Drizzle (hide the SQL the course grades)
UI	Tailwind v4, shadcn/ui, React Bits, Sonner, Recharts, Leaflet + OpenStreetMap	Accessible primitives you own as source, a few deliberate animated components, maps without API keys, and charts for the dashboards.	

Layer	Choice	Why	Rejected alternatives
Validation	Zod (app) + CHECK constraints (DB)	Defence in depth. The DB is the final authority.	

The consistency question (examiners will ask)

Postgres is the *source of truth*. Qdrant is a *derived index*. A complaint is committed in SQL first, then embedded and upserted to Qdrant. If Qdrant is down, the complaint is still saved with `vector_synced_at = NULL`, and a reconcile job re-indexes every unsynced row. Duplicate decisions are stored back in SQL (`duplicate_match`). Writing to two stores is a deliberate trade-off. Explain it with this sentence.

6 Local architecture



Everything runs on one laptop: `docker compose up` starts Postgres and Qdrant, then `npm run dev` starts Next.js. No cloud accounts or API keys are needed. The embedding model downloads once on first use and is cached.

7 Final schema v2

15 tables: 13 domain tables and 2 support tables. NEW marks a table that doesn't exist in your live DB today. Names are snake_case and singular.

Table	Columns (PK underlined, FK →)	Key constraints
department	department_id, department_name, contact_email, description, created_at	UNIQUE(name); CHECK email ~ regex
ward	ward_id, ward_number, ward_name, zone, ward_population, created_at	UNIQUE(ward_number), UNIQUE(ward_name); CHECK population ≥ 0
app_user (was "User")	user_id, name, email, password_hash, role, department_id→department, ward_id→ward, supervisor_id→app_user, is_active, created_at	UNIQUE(email); CHECK role IN (CITIZEN, STAFF, ADMIN); role-consistency CHECK (staff ⇒ dept & ward; citizen ⇒ neither); supervisor ≠ self
user_phone NEW	user_id→app_user, phone_number	Maps the multivalued Phone_Number; CHECK phone ~ '^\+?[0-9]{7,15}$'; ON DELETE CASCADE
complaint_category NEW	category_id, category_name, department_id→department, sla_hours	UNIQUE(name); CHECK sla_hours BETWEEN 1 AND 720
complaint	complaint_id, title, description, category_id→complaint_category, status, priority, address, latitude, longitude, created_at, updated_at, sla_deadline, resolved_at, filed_by→app_user, assigned_to→app_user, department_id→department, ward_id→ward, duplicate_of→complaint, upvote_count, search_tsv (generated), vector_synced_at	status CHECK (8 values); priority CHECK; sla_deadline ≥ created_at; resolved_at ≥ created_at; DUPLICATE ⇒ duplicate_of NOT NULL; duplicate_of ≠ self; lat BETWEEN −90 AND 90, long BETWEEN −180 AND 180; upvote_count ≥ 0
status_transition NEW	from_status, to_status, actor_role	The state machine as data (reference table) that a trigger checks
status_history (weak)	complaint_id→complaint, history_no, old_status, new_status, changed_by→app_user, remarks, changed_at	Identifying FK ON DELETE CASCADE; old ≠ new
escalation_policy NEW	escalation_level, target_role, hours_after_breach, label	Level 1 = dept head (supervisor), 2 = ward admin, 3 = municipal admin
escalation (weak)	complaint_id→complaint, escalation_level→escalation_policy, escalated_from→app_user, escalated_to→app_user, reason, escalated_at, resolved_at	Partial key = level; resolved_at ≥ escalated_at
upvote	complaint_id→complaint, user_id→app_user, upvoted_at	M:N associative; trigger blocks upvoting your own complaint

Table	Columns (PK underlined, FK →)	Key constraints
duplicate_match NEW	complaint_id→complaint, matched_complaint_id→complaint, similarity_score, review_status, reviewed_by→app_user, detected_at, reviewed_at	score BETWEEN 0 AND 1; complaint ≠ matched; status IN (PENDING, CONFIRMED, REJECTED)
notification	notification_id, user_id→app_user, complaint_id→complaint, type, message, read_status, sent_at	type IN (ASSIGNMENT, STATUS_UPDATE, ESCALATION, DUPLICATE_FLAG, SYSTEM)
user_session NEW	token_hash, user_id→app_user, created_at, expires_at, user_agent	Stores only the SHA-256 of the cookie token; expires_at > created_at
schema_migrations NEW	version, applied_at	Tracks which migration files have run

Normalization argument (viva-ready)

In Lab 4, `complaint.category` was free text, and SLA hours and the owning department depend on the category, not on the complaint. That is a transitive dependency (complaint → category → sla_hours, department), so the table is not in 3NF. Moving them into `complaint_category` fixes it.

`complaint.department_id` stays because it records the *currently handling* department, which an admin can reassign away from the category default. It is a fact about the complaint, not about the category.

State machine (status_transition rows)

```
OPEN → ASSIGNED (system/admin)
ASSIGNED → IN_PROGRESS (staff)
IN_PROGRESS → RESOLVED (staff)
RESOLVED → CLOSED (citizen, or auto after 7 days)
RESOLVED → REOPENED (citizen)
REOPENED → IN_PROGRESS (staff)
OPEN|ASSIGNED|IN_PROGRESS|REOPENED → ESCALATED (system)
ESCALATED → IN_PROGRESS (staff/admin)
OPEN|ASSIGNED → DUPLICATE (staff/admin)
any open state → CLOSED (admin)
```

Diagram work (rubric items 1 and 2)

- ☐ **Chen ER v2** (start from your Lab 4 diagram): add the CATEGORY entity (Complaint *belongs to* Category N:1; Category *handled by* Department N:1), the *supervises* recursive relationship on USER, USER *works in* WARD, the NOTIFICATION *about* COMPLAINT relationship, and DUPLICATE MATCH as an M:N recursive relationship on COMPLAINT with attributes (score, review status). Keep Phone_Number multivalued, Location composite and Resolution_Time derived. STATUS_HISTORY and ESCALATION stay weak with identifying relationships and partial keys history_no and escalation_level. Mark participation (double lines) and cardinalities on every relationship.
- ☐ **Relational schema v2**: redo the Excalidraw file as the 15 tables above, with PK underlined, FK arrows pointing to the referenced PK, and a legend for UNIQUE and CHECK. Export PNG and SVG to `docs/`.
- ☐ Add a one-page **ER → relational mapping** note to the docs: strong entity → table; weak entity → composite PK + CASCADE; 1:N → FK on the N side; M:N (upvote, duplicate match) → associative table; multivalued → separate table; composite → flattened columns; derived → computed in a view.

8 Triggers, functions, procedures, views

Rubric item 8 (6 marks) asks for procedure/function, trigger and CTE. This list goes well past the minimum, and every object is used by a real feature.

Object	Type	What it does / why it's there
`trg_complaint_before_insert`	Trigger, BEFORE INSERT	Copies department_id from the category and sets `sla_deadline = created_at + sla_hours`. Status starts as OPEN.
`trg_complaint_after_insert`	Trigger, AFTER INSERT	Calls `fn_pick_assignee()`, moves the complaint to ASSIGNED, writes history row #1, and notifies the staff member.
`trg_complaint_validate_status`	Trigger, BEFORE UPDATE OF status	Rejects transitions not in `status_transition` for the actor's role. Sets resolved_at and updated_at.
`trg_complaint_status_audit`	Trigger, AFTER UPDATE OF status	Inserts into `status_history` (next history_no) and notifies the citizen.
`trg_upvote_guard` / `trg_upvote_count`	Triggers on upvote	Blocks self-upvotes. Keeps `complaint.upvote_count` in sync on INSERT/DELETE (denormalized counter).
`trg_escalation_notify`	Trigger, AFTER INSERT ON escalation	Notifies the escalation target, the assigned staff member and the citizen.
`fn_pick_assignee(dept, ward)`	Function	Picks the active staff member in that department and ward with the fewest open complaints (aggregate + ORDER BY + LIMIT).
`fn_escalation_target(cid, level)`	Function, recursive CTE	Walks the `supervisor_id` chain upward to find the right person for that escalation level.
`sp_escalate_overdue()`	Procedure	The SLA sweep. Locks overdue rows with `FOR UPDATE SKIP LOCKED` (so two runs can never double-escalate), inserts escalations and sets ESCALATED. Run by pg_cron every 5 minutes and by the admin's "Run sweep now" button.
`sp_change_status(cid, new, actor, remarks)`	Procedure, transaction	Status change, history, notification: atomic, all or nothing.
`sp_confirm_duplicate(cid, mid, reviewer)`	Procedure, transaction	Marks the match CONFIRMED, sets duplicate_of and status DUPLICATE, and moves upvotes to the canonical complaint (`INSERT … ON CONFLICT DO NOTHING`).
`fn_sla_compliance(dept, from, to)`	Function, CTE	Returns the percentage of complaints resolved within SLA. Feeds the admin analytics.
`fn_register_citizen()` / `fn_authenticate()`	Functions, SECURITY DEFINER	Password hashing and checking inside Postgres with `pgcrypto`: `crypt(pw, gen_salt('bf', 10))`.

Object	Type	What it does / why it's there
`vw_complaint_full_detail`, `vw_sla_breach_candidates`, `vw_escalation_chain`, `vw_department_workload`	Views (from Lab 4, updated)	Keep your Lab 4 views. Update status names and columns, and set `security_invoker = true` so RLS still applies through them.
`mv_public_ward_stats`	Materialized view	Open/resolved counts, average resolution hours and SLA % per ward and department for the public dashboard. pg_cron refreshes it every 10 minutes.
Indexes	B-tree, composite, partial, GIN	Keep the Lab 4 indexes. Add the partial index `(sla_deadline) WHERE status IN (open states)` for the sweep and the GIN index on `search_tsv`. Show `EXPLAIN ANALYZE` before and after in the docs (you did this in Lab 4; reuse the story).

```
-- Skeleton of the SLA sweep. The real version lives in db/migrations/004_procedures.sql
CREATE OR REPLACE PROCEDURE sp_escalate_overdue()
LANGUAGE plpgsql AS $$
DECLARE r RECORD; v_level INT; v_target INT;
BEGIN
  FOR r IN
    SELECT c.complaint_id, c.assigned_to, c.sla_deadline,
           COALESCE((SELECT MAX(e.escalation_level) FROM escalation e
                     WHERE e.complaint_id = c.complaint_id), 0) AS cur_level
    FROM complaint c
    WHERE c.status IN ('OPEN','ASSIGNED','IN_PROGRESS','REOPENED','ESCALATED')
      AND c.sla_deadline < now()
    FOR UPDATE OF c SKIP LOCKED              -- concurrent sweeps never double-escalate
  LOOP
    v_level := r.cur_level + 1;
    CONTINUE WHEN v_level > 3;
    CONTINUE WHEN now() < r.sla_deadline + (SELECT make_interval(hours => p.hours_after_breach)
                                            FROM escalation_policy p WHERE p.escalation_level =
v_level);
    v_target := fn_escalation_target(r.complaint_id, v_level);
    INSERT INTO escalation (complaint_id, escalation_level, escalated_from, escalated_to, reason)
    VALUES (r.complaint_id, v_level, r.assigned_to, v_target,
            format('SLA breached by %s h', round(extract(epoch FROM now() - r.sla_deadline) / 3600)));
    UPDATE complaint SET status = 'ESCALATED'
    WHERE complaint_id = r.complaint_id AND status <> 'ESCALATED';
  END LOOP;
END $$;

SELECT cron.schedule('sla-sweep', '*/5 * * * *', $$CALL sp_escalate_overdue()$$);
```

Demo trick: don't wait 72 hours

Seed about 15 complaints whose `sla_deadline` is already in the past at different breach ages, plus one "Demo, 2-minute SLA" category. In the demo, click "Run sweep now" and the escalations appear live with their notifications and the recursive escalation chain.

9 Query catalogue

Rubric items 6 and 7 (8 marks). Each query is a file in `db/queries/` with a header comment (purpose, concept demonstrated, expected output), and each runs from the admin **DB Lab** page.

DML (item 6)

- Seed: 10 Bengaluru wards, 7 departments, 14 categories with realistic SLAs (garbage 24 h, pothole 72 h, fallen tree 12 h…), ~60 users across 3 roles, ~300 complaints spread over 90 days, plus upvotes, history and escalations
- `INSERT … RETURNING complaint_id` (file a complaint)
- `INSERT … ON CONFLICT DO NOTHING` (upvote toggle, duplicate upvote merge)
- `UPDATE … FROM` (bulk reassign a ward's open complaints to another staff member)
- `DELETE … USING` (purge expired sessions; withdraw an OPEN complaint)
- Transactional status change with history (`sp_change_status`)

Joins

- 5-way INNER/LEFT join: complaint full detail (from Lab 4)
- LEFT JOIN + COUNT: departments including those with zero complaints
- Self-join: candidate duplicate pairs, with both titles side by side
- Anti-join (`NOT EXISTS`): staff with no open assignments
- FULL OUTER JOIN: wards × categories coverage matrix

Nested & correlated

- Nested (HAVING subquery): wards whose open count exceeds the average across all wards
- Nested (`IN`): citizens who filed in more than 3 distinct categories
- Correlated: complaints whose resolution time exceeds *their own department's* average
- Correlated (`EXISTS`): complaints escalated twice or more in the last 30 days
- GROUP BY / HAVING: departments with more than 5 complaints past SLA (from your proposal)

Recursive & CTE

- `WITH RECURSIVE`: duplicate chain to its root (canonical) complaint, with a `CYCLE` guard
- `WITH RECURSIVE`: supervisor chain (staff → dept head → ward admin → commissioner)
- Recursive date series: daily filed vs resolved counts (no gaps on quiet days)
- Multi-CTE + window functions: rank wards by SLA compliance, running totals, `LAG` week-over-week change
- Full-text: `search_tsv @@ websearch_to_tsquery()` ranked by `ts_rank`, which sets up the "keyword vs semantic" comparison

```
-- Correlated: slower than their own department's average
SELECT c.complaint_id, c.title, d.department_name,
       round(extract(epoch FROM c.resolved_at - c.created_at) / 3600, 1) AS hours_taken
FROM complaint c
JOIN department d USING (department_id)
WHERE c.resolved_at IS NOT NULL
  AND c.resolved_at - c.created_at > (SELECT avg(c2.resolved_at - c2.created_at)
                                      FROM complaint c2
                                      WHERE c2.department_id = c.department_id
                                        AND c2.resolved_at IS NOT NULL);

-- Recursive: follow duplicate_of to the canonical complaint
WITH RECURSIVE chain AS (
  SELECT complaint_id, duplicate_of, 0 AS depth FROM complaint WHERE complaint_id = $1
  UNION ALL
  SELECT c.complaint_id, c.duplicate_of, ch.depth + 1
  FROM complaint c JOIN chain ch ON c.complaint_id = ch.duplicate_of
) CYCLE complaint_id SET is_cycle USING path
SELECT * FROM chain ORDER BY depth;
```

10 Vector database design (Qdrant)

Rubric items 4 and 9 are worth 11 marks together, the largest single block, so each operation is tied to a visible product feature.

Collection: `complaints`

- Vector: 384-d, **Cosine** distance, HNSW index (default m = 16)
- Point ID = `complaint_id` (the SQL primary key; Qdrant accepts unsigned integer IDs)
- Embedded text: `title + ". " + description`, normalized
- Payload: complaint_id, ward_id, category_id, department_id, status, is_open, created_at, lat, lon
- Payload indexes: ward_id, category_id (integer); status (keyword); is_open (bool); created_at (datetime)
- Created by `scripts/qdrant-setup.ts`, idempotent and versioned in git

Duplicate-detection strategy

- A **live check as the citizen types** (debounced 600 ms, after 25+ characters): top 5 neighbours
- Filter: same ward, `is_open = true`, created in the last 60 days
- `score_threshold` = a calibrated value (below), not a guess
- The UI offers "This looks like #1042 (similarity 0.87): upvote it instead?"
- On submit, candidates above the threshold become `duplicate_match` rows (PENDING) in the staff review queue

CRUD → real features

Op	Qdrant call	Where it happens in the product
Create	`createCollection`, `createPayloadIndex`, `upsert`	Setup script; every new complaint is embedded and upserted after the SQL commit; `npm run qdrant:reindex` rebuilds everything from SQL
Read	`query` (vector + filter + threshold), `retrieve`, `query` by point id ("recommend"), `count`, `scroll`	Live duplicate suggestions; "Similar complaints" panel for staff; Vector Console: point viewer, collection stats, search playground showing scores
Update	`setPayload`, `upsert` (re-embed)	Every status change, reassignment or duplicate confirmation syncs the payload (`is_open` flips to false on RESOLVED, CLOSED or DUPLICATE). Editing an OPEN complaint's text re-embeds it.
Delete	`delete` (by id and by filter)	A citizen withdraws an OPEN complaint; an admin purges test data; the Vector Console has delete-by-filter (e.g. all points of a demo ward)

```
// src/lib/vector.ts (abridged)
const extractor = await pipeline('feature-extraction', 'Xenova/all-MiniLM-L6-v2');   //
@huggingface/transformers
export async function embed(text: string) {
  const out = await extractor(text, { pooling: 'mean', normalize: true });
  return Array.from(out.data as Float32Array);                                    // 384 numbers
}
export const findDuplicates = async (text: string, wardId: number) =>
  qdrant.query('complaints', {
    query: await embed(text),
    filter: { must: [{ key: 'ward_id', match: { value: wardId } },
                     { key: 'is_open', match: { value: true } },
                     { key: 'created_at', range: { gte: daysAgo(60) } }] },
    limit: 5, score_threshold: DUPLICATE_THRESHOLD, with_payload: true,
  });
```

Calibrate the threshold. This also gives you a resume line

Write 40 paraphrase pairs that should match ("huge pothole near Silk Board signal" / "big crater on road by Silk Board junction") and 40 pairs that shouldn't (same area, different problem). `scripts/eval-duplicates.ts` prints precision and recall at thresholds 0.60–0.90. Pick the best F1, then put the table and the chosen value in `docs/vector-db.md`. Also show 3 queries where full-text search finds nothing but vector search finds the duplicate. That's your "why vectors" proof.

11 Authentication & RBAC in the database

Authentication

- Register/login go through `fn_register_citizen` / `fn_authenticate`. bcrypt hashing via `pgcrypto` runs inside Postgres, so the plaintext password never touches a table.
- Session = 32 random bytes in an **httpOnly, SameSite=Lax** cookie (Secure when deployed). Only its SHA-256 is stored in `user_session`. Logout deletes the row.
- `proxy.ts` only checks that the cookie exists and redirects (optimistic). Real checks happen in a server-side data-access layer, per Next.js guidance.
- Zod validation on every input. Staff and admin accounts are created only by admins. Citizens self-register.
- Login page shows demo accounts (one per role) when `DEMO_MODE=true`.

RBAC inside Postgres

- Group roles `civic_anon`, `civic_citizen`, `civic_staff`, `civic_admin` with table-level `GRANT`s
- The app logs in as `civic_app` (NOINHERIT, no table rights of its own). Every request runs `BEGIN; SET LOCAL ROLE civic_<role>; set_config('app.user_id', …)`
- **Row-Level Security** on complaint, notification, upvote, duplicate_match and user_session
- Trigger functions that write for other users (notifications, history) are `SECURITY DEFINER` with a pinned `search_path`
- Proof for the demo: in psql, `SET ROLE civic_citizen` plus a user id shows that only that citizen's rows come back

```
ALTER TABLE complaint ENABLE ROW LEVEL SECURITY;
CREATE POLICY citizen_read  ON complaint FOR SELECT TO civic_citizen USING (filed_by = app_user_id());
CREATE POLICY staff_read    ON complaint FOR SELECT TO civic_staff   USING (department_id = app_dept()
AND ward_id = app_ward());
CREATE POLICY staff_update  ON complaint FOR UPDATE TO civic_staff   USING (assigned_to =
app_user_id());
CREATE POLICY admin_scope   ON complaint FOR ALL    TO civic_admin   USING (app_ward() IS NULL OR
ward_id = app_ward());

// src/lib/db.ts: every query runs inside the caller's database role
export const withRole = (s: Session, fn: (tx: TransactionSql) => Promise<T>) =>
  sql.begin(async (tx) => {
    await tx.unsafe(`SET LOCAL ROLE ${ROLE_MAP[s.role]}`);        // whitelisted constant, never user
input
    await tx`SELECT set_config('app.user_id', ${String(s.userId)}, true)`;
    return fn(tx);
  });
```

Permission matrix

Capability	Public	Citizen	Staff	Ward admin	Municipal admin
View aggregated public dashboard and map	✓	✓	✓	✓	✓
File complaint, upvote, track own, confirm/reopen own	–	✓	–	–	–
See and work the queue in own department + ward	–	–	✓	✓	✓
Review duplicate matches	–	–	✓ (own dept)	✓	✓
Reassign, close, run SLA sweep, view escalations	–	–	–	✓ (own ward)	✓ (all)
Manage users and roles, DB Lab, Vector Console	–	–	–	read	✓

There are three database roles (citizen, staff, admin), matching the proposal. A ward admin is an admin with a ward_id. The municipal admin (commissioner) is an admin whose ward_id is NULL. RLS expresses that scope.

12 Next.js application

Pages

Area	Route	What's on it
Public	`/`	Landing page: live counters (resolved this month, avg resolution, SLA %), how it works, recent resolutions. The only place for expressive React Bits motion.
	`/dashboard`	Ward map (Leaflet + OSM) shaded by open complaints; per-ward and per-department stats from `mv_public_ward_stats`
	`/search`, `/track/[id]`	Full-text search of public complaint titles; public status timeline
Auth	`/login`, `/register`	Forms with Zod errors, demo-account chips
Citizen	`/report`	3 steps: describe (live duplicate suggestions with scores) → locate (pin on map, ward auto-selected) → review and submit. "Upvote existing instead" shortcut.
	`/me/complaints`, `/me/complaints/[id]`	Own complaints with SLA countdown, status timeline (status_history), escalation badges, confirm/reopen actions
	`/notifications`	Inbox; mark read
Staff	`/staff/queue`	Queue sorted by time-to-breach (green/amber/red), filters, bulk actions; transition buttons show only legal next states
	`/staff/complaints/[id]`	Detail, similar complaints (vector "recommend"), history, remarks
	`/staff/duplicates`	Side-by-side pair review with similarity score → confirm or reject (`sp_confirm_duplicate`)
Admin	`/admin`	Analytics: SLA compliance by department (bar), filed vs resolved (line, recursive date series), ward ranking (window functions), escalation funnel
	`/admin/escalations`	Escalation board, chain view (recursive CTE), "Run SLA sweep now"
	`/admin/users`	Create staff and admins, assign department, ward and supervisor, deactivate
	`/admin/db-lab`	**Evaluator view:** each rubric query or procedure with its SQL, a Run button, a results table and `EXPLAIN ANALYZE`
	`/admin/vector`	**Vector Console:** collection info, point count, search playground, retrieve/update/delete point, reindex, link to the Qdrant dashboard

API (`/api/v1`, JSON, Zod-validated, role-checked)

`POST /auth/register · POST /auth/login · POST /auth/logout · GET /auth/me`
`GET /complaints?status&ward&category&q&page · POST /complaints · GET /complaints/:id · PATCH /complaints/:id (edit while OPEN) · DELETE /complaints/:id (withdraw)`
`POST /complaints/:id/status · POST /complaints/:id/assign · POST|DELETE /complaints/:id/upvote · GET /complaints/:id/similar`
`POST /duplicates/check · GET /duplicates?status=PENDING · POST /duplicates/:cid/:mid/review`
`GET /escalations · POST /escalations/sweep · GET /notifications · POST /notifications/read`
`GET /stats/public · GET /stats/admin · GET /admin/users · POST /admin/users · GET /lab/queries ·`

```
POST /lab/queries/:name/run
GET /vector/collection · GET /vector/points/:id · PATCH /vector/points/:id · DELETE
/vector/points · POST /vector/reindex · GET /health
```

Server Actions drive the forms (built-in origin check). Route Handlers make the REST surface explicit for the "Backend/API" mark and for testing with curl or Bruno. Document them in `docs/api.md` with example requests.

Design direction (starting point for `/impeccable init`)

Identity: civic signage, not a SaaS template

Warm off-white paper, deep ink navy, **signal amber** reserved for SLA risk, municipal green for resolved. Public Sans as the typeface (an open-source font made for government services, and it avoids the Inter look Impeccable flags) and IBM Plex Mono for IDs and timestamps. Dense, calm work screens; the expressive moments are only on the landing page and the dashboard. Dark mode and WCAG AA contrast. Fully usable on a phone, because citizens report from the street.

Where motion belongs

React Bits on the public pages only: one text reveal for the hero headline, count-up numbers for the stats, an animated list for the live "recently resolved" feed, and at most one subtle background. Inside work queues: no decorative motion, only functional transitions (status change, toast via Sonner, SLA bar). Emil Kowalski's skills review timing and easing.

13 Toolkit: where each repo is used

These are **agent tooling** (skills and plugins for Claude Code). They change *how* the app gets built and reviewed. None of them ship inside CivicFix, except React Bits components, which are copied in as source. Plugin installs are slash commands **you** run in Claude Code.

Repo	Install	Use it for (and when)
pbakaus/impeccable	`npx impeccable install` in the repo, then `/impeccable init`	**Primary design system for the app UI.** `init` writes PRODUCT.md (audience: citizens, staff, admins). Run `shape` before each surface, `craft` to build, `critique` / `audit` after, and `harden` + `polish` in Phase 7. Its 62 detector rules catch generic "AI design".
VismayHS/taste-skill-cc	`npx skills add https://github.com/VismayHS/taste-skill-cc --skill "design-taste-frontend"` (+ `minimalist-ui`)	The default skill is for *public-facing pages* (its README says it is not for dashboards), so use it for `/` and `/dashboard` art direction. Use `minimalist-ui` as the house style for the staff and admin screens.
VismayHS/skills-frontend-cc (Emil Kowalski's skills)	`npx skills add https://github.com/VismayHS/skills-frontend-cc`	`emil-design-eng` + `animate` for motion; `pick-ui-library` before adding dependencies; `ask-sonner` for toasts; `break-ui` in Phase 7 (long names, empty lists, huge counts); `mobile-native` for the citizen flow; `review-animations` before the demo.
React Bits (DavidHDev)	Add the `@react-bits` registry to `components.json`, then `npx shadcn@latest add @react-bits/<Name>-TS-TW`	3–4 components on public pages (see §12). No version to pin: components are copied as source at their latest revision. License is MIT + Commons Clause (fine for this use).
dietrichgebert/ponytail	`/plugin marketplace add DietrichGebert/ponytail`, then `/plugin install ponytail@ponytail`	Always on at `full`: keeps the code lean and readable, so the viva is easier when there's less code to defend. It never cuts validation or security. `/ponytail-review` before every merge; `/ponytail-audit` once at the end. Its status band doesn't show in the VS Code chat panel, only in the terminal and desktop app.
affaan-m/ECC	`/plugin marketplace add https://github.com/affaan-m/ECC`, then `/plugin install ecc@ecc`. **One install path only.**	Planning, test-driven development and security-review agents, plus **AgentShield** to scan for leaked secrets and risky config before you push. It's large (311 skills) and overlaps with ponytail and Claude Code's built-ins. Use its security review and TDD flow; ignore the rest.

Repo	Install	Use it for (and when)
ruvnet/ruflo	Optional. `/plugin marketplace add ruvnet/ruflo`, then `/plugin install ruflo-core@ruflo`	Swarm orchestration for parallel independent tasks (e.g. seed generation + query files + tests at once). Honest take: this project doesn't need it, and Claude Code's own subagents cover parallel work. Its mods run with your account's permissions and aren't sandboxed, so read the code before installing. Recommended: skip it, or use the core plugin only.

Install order and hygiene

1) impeccable (project scope) → 2) skills-frontend-cc and taste-skill → 3) ponytail → 4) ECC, minimal → 5) ruflo only if wanted. Commit the generated `PRODUCT.md` / `DESIGN.md`, since they document design intent and count toward documentation. Don't commit agent caches or local settings: add them to `.gitignore`.

14 Repository layout & local setup

```
CivicFix/
├─ README.md                  # screenshots, features, setup in 5 commands, rubric map, team
├─ docker-compose.yml         # postgres (with pg_cron) + qdrant
├─ .env.example               # every variable, no secrets
├─ db/
│  ├─ docker/Dockerfile       # FROM postgres:17 + postgresql-17-cron
│  ├─ migrations/             # 001_schema · 002_reference_data · 003_functions · 004_procedures
│  │                          # 005_triggers · 006_views · 007_rbac_rls · 008_cron · 009_indexes
│  ├─ seed/                   # deterministic seed (wards, depts, users, 300 complaints)
│  ├─ queries/                # one .sql per rubric query, with header comments
│  └─ tests/                  # SQL assertion tests (DO blocks that RAISE on failure)
├─ scripts/                   # migrate.ts · qdrant-setup.ts · reindex.ts · eval-duplicates.ts
├─ src/
│  ├─ app/(public) (auth) (citizen) (staff) (admin)   api/v1/…
│  ├─ components/             # ui/ (shadcn), bits/ (React Bits), domain components
│  ├─ lib/                    # db.ts (withRole) · auth.ts · vector.ts · validators.ts
│  └─ proxy.ts
├─ tests/                     # vitest (lib) · playwright (e2e per role)
├─ docs/                      # er-v2.png · relational-v2.png · *.excalidraw · technology-
justification.md
│                             # vector-db.md · queries.md · api.md · rbac.md · demo-script.md ·
CONTRIBUTIONS.md
└─ .github/workflows/ci.yml   # lint · typecheck · migrations + SQL tests on postgres service · e2e
```

From clone to running app

```
git clone https://github.com/VismayHS/CivicFix && cd CivicFix
cp .env.example .env.local          # set local passwords
docker compose up -d                # Postgres :5432, Qdrant :6333
npm install
npm run db:migrate && npm run db:seed
npm run qdrant:setup && npm run qdrant:reindex
npm run dev                         # http://localhost:3000
npm test && npm run test:sql && npm run test:e2e
```

Postgres needs `shared_preload_libraries=pg_cron` and `cron.database_name=civicfix`, set via the compose `command`. Pin the Qdrant image to an exact release tag rather than `latest`. Postgres and Qdrant data live in named Docker volumes, so `docker compose down -v` gives a clean slate.

Git & GitHub practice (rubric item 13)

- Fix the README heading ("CivicFix-" → "CivicFix"). Add a description, topics (`nextjs postgresql qdrant vector-search dbms`) and a screenshot.
- One GitHub issue per rubric item or feature; a Project board (To do / Doing / Done); milestones per phase.
- Feature branches → pull requests with short descriptions. Each person merges their own. Conventional commit messages (`feat(db): add sp_escalate_overdue`).
- Tags: `v0.1-db`, `v0.2-app`, `v1.0-demo`. `CONTRIBUTIONS.md` maps each person to files and PRs.
- Never commit `.env*` (except `.env.example`). Run AgentShield or a secret scan before pushing.

15 Phased build plan

Owners follow your proposal: ASHVIN database layer · VISMAY app, auth, API, vector · BOTH. Each phase ends with a merged PR and a working build. Day counts assume ~3–4 focused hours a day. Add your evaluation date here: _____.

Phase	Tasks	Done when
0 · Setup ~1 day	☐ Repo skeleton, Next.js 16 + TS + Tailwind v4 + shadcn, ESLint/Prettier V ☐ docker-compose (Postgres 17 + pg_cron, Qdrant), .env.example, migrate script A ☐ Install toolkit (§13), run `/impeccable init` → PRODUCT.md V ☐ Issues and Project board for all 14 rubric items B	`docker compose up` and `npm run dev` both work on both laptops
1 · Design fix ~1 day	☐ Freeze schema v2 (§7); write the ER→relational mapping note B ☐ Redraw the Chen ER diagram v2 A · Relational schema v2 in Excalidraw V	Diagrams and table list match 1:1 (checked against the §4 list)
2 · DB core ~2–3 days	☐ Migrations 001–002: tables, constraints, reference data (status_transition, escalation_policy) A ☐ 003–005: functions, procedures, triggers (§8) A ☐ 006: views + materialized view; 009: indexes, with EXPLAIN before/after A ☐ Deterministic seed with overdue complaints for the demo A ☐ SQL tests: illegal transition rejected, SLA deadline computed, self-upvote blocked, sweep idempotent A	`npm run db:migrate && db:seed && test:sql` pass from an empty DB
3 · Queries ~1–2 days	☐ All §9 queries as files with headers and expected output A ☐ 007: roles, GRANTs, RLS policies + an RLS isolation test A ☐ 008: pg_cron jobs (sweep, MV refresh, session purge, auto-close) A	docs/queries.md generated; RLS test passes
4 · App + auth ~2 days	☐ `lib/db.ts` withRole, `lib/auth.ts` sessions, proxy.ts, login/register UI V ☐ App shell per role, nav, notifications bell V ☐ /api/v1 handlers for complaints and status, Zod schemas V	Each demo user logs in and sees only their own scope
5 · Vector ~2 days	☐ Embedder + qdrant-setup + reindex scripts V ☐ Live duplicate check in /report, upsert on create, payload sync on status change, delete on withdraw V ☐ duplicate_match flow + sp_confirm_duplicate wiring B ☐ Threshold calibration script + results table V	Paraphrased complaint is flagged; all CRUD paths visible in the Vector Console
6 · Role UIs ~3 days	☐ Citizen: report wizard, my complaints, timeline V ☐ Staff: SLA queue, detail, duplicate review V ☐ Admin: analytics, escalations, users, **DB Lab**, **Vector Console** V (queries supplied by A) ☐ Public: landing (React Bits), dashboard map, search, track V	Full demo script (§16) runs without touching psql

Phase	Tasks	Done when
7 · Polish ~2 days	☐ `/impeccable audit`, `harden`, `polish`; `break-ui`; keyboard and screen-reader pass; mobile pass V ☐ Playwright e2e for 3 role journeys; CI green B ☐ README, docs/, CONTRIBUTIONS.md, demo script, screenshots B ☐ `/ponytail-audit` + ECC security review; fix the top findings B ☐ Viva drills (§16) B	Tag `v1.0-demo`; a dry-run demo under 8 minutes
8 · Deploy later	See §17. Not needed for evaluation.	

16 Testing, demo script & viva prep

Tests

- **SQL** (`db/tests`): constraint violations raise errors; each trigger fires; the sweep escalates exactly once per level; RLS isolation (citizen A can't read B's rows); duplicate confirm moves upvotes.
- **Unit** (Vitest): validators, session helpers, embed() returns 384 normalized numbers.
- **E2E** (Playwright): citizen files a complaint and sees a duplicate suggestion; staff resolves it; admin runs the sweep and sees an escalation.
- **CI**: GitHub Actions with a Postgres service container runs migrations, seed and all tests on every PR.

Demo script (≈8 min)

1. Public dashboard and ward map (materialized view)
2. Citizen: type a paraphrase of an existing complaint → live suggestion with score → upvote instead. File a new one → auto-assigned by trigger → notification
3. Staff: queue sorted by SLA, move to IN_PROGRESS, try an illegal transition (rejected by the DB), confirm a duplicate pair
4. Admin: "Run SLA sweep now" → escalations and the chain (recursive CTE); analytics charts
5. DB Lab: run nested, correlated and recursive queries with EXPLAIN
6. Vector Console + Qdrant dashboard: create, read, update, delete live
7. psql: `SET ROLE civic_citizen` → RLS filters rows
8. GitHub: issues, PRs, split commit history, docs folder

Likely viva questions (prepare a 2-line answer each)

- Why Qdrant over pgvector? How do you keep the two stores consistent?
- Why cosine? What does a score of 0.87 mean? How did you pick the threshold?
- Trigger vs procedure vs function: why is the sweep a procedure?
- What does `SKIP LOCKED` prevent? What happens if two sweeps overlap?
- Why are status_history and escalation weak entities? What's the partial key?
- Show the 3NF problem in the old category column.
- How does RLS know who the user is? Why `SET LOCAL` and not `SET` ?
- What is `SECURITY DEFINER` , and what's the risk?
- Why is the composite index (category, ward_id, created_at DESC) in that column order?
- Full-text vs vector search: show a case where only one works.

Modification drills (practise live, 5 min each)

- Add a `priority` rule: CRITICAL complaints get half the SLA (trigger change)
- New query: top 3 wards by escalations this month
- Add a status `ON_HOLD` (CHECK + transition rows + UI button)
- Add a 4th role `DEPT_HEAD` with its own RLS policy
- Restrict duplicate search to within 500 m (payload geo filter in Qdrant)

17 Future: deployment plan

Not needed for the evaluation. Keep the code deploy-ready (environment variables only, no hard-coded localhost URLs) so this becomes a one-afternoon job later.

Piece	Service	Notes (verified Oct 2026)
Next.js app	Vercel (Hobby)	Native Next.js host. Hobby cron jobs run **at most once a day**, so keep the SLA sweep in pg_cron, not Vercel Cron.
PostgreSQL	Supabase (free)	Supports pg_cron (Supabase Cron runs SQL and database functions on a schedule) and custom roles + RLS. Use the direct Postgres connection, not supabase-js or Supabase Auth, so your own auth and RBAC stay the graded work. Free projects **pause after 7 days** without activity. Use the transaction pooler with prepared statements disabled.

Piece	Service	Notes (verified Oct 2026)
Vector DB	Qdrant Cloud (free cluster)	1 node, 1 GB RAM, 4 GB disk. **Suspended after 1 week unused, deleted after 4 weeks** unless reactivated. Qdrant Cloud Inference serves `all-minilm-l6-v2` (384-d) free, so embeddings move server-side with no model change.
Keep-alive	GitHub Actions schedule	A daily workflow hits `/api/health`, which touches both databases, keeping the free tiers awake before demos and interviews.
Region	us-east-1 for all three	Co-locating the app, DB and vector DB minimizes server-to-server latency. Only the browser hop is long-haul.

18 Resume & portfolio

Fill the bracketed numbers from your own measurements. Never put an unmeasured number on a resume.

CivicFix: Civic Grievance Platform · Next.js, PostgreSQL, Qdrant · `github.com/VismayHS/CivicFix`

- Built a full-stack grievance redressal platform with 3 role-based portals. Detects duplicate complaints semantically using MiniLM sentence embeddings and filtered cosine k-NN search in Qdrant, tuned to a [0.xx] threshold for [xx]% precision / [xx]% recall on [80] labelled pairs.
- Designed a 15-table normalized PostgreSQL schema with database-native workflow: PL/pgSQL triggers for SLA deadlines, auto-assignment and audit history, and a `pg_cron`-scheduled escalation procedure using `FOR UPDATE SKIP LOCKED` for safe concurrent runs.
- Enforced access control inside the database with PostgreSQL roles and Row-Level Security, bound per request via transaction-scoped `SET LOCAL ROLE`, plus pgcrypto password hashing and hashed session tokens.
- Shipped with CI running migrations, SQL assertion tests and Playwright end-to-end tests on every PR. Indexing cut dashboard query time from [x] ms to [y] ms (EXPLAIN ANALYZE).

Make it explainable

- README opens with a 60-second GIF of the duplicate-detection moment, an architecture diagram (§6) and "run it in 5 commands".
- A short write-up in `docs/` (or a blog post): "Catching duplicate civic complaints with vector search: what full-text search misses". Include the threshold table.
- Pin the repo on your GitHub profile. Add a LinkedIn project entry with the GIF.

19 Risks & mitigations

Risk	Mitigation
Viva: can't explain AI-written code	Each owner writes or rewrites their part, keeps code minimal (ponytail), and rehearses the §16 drills. CONTRIBUTIONS.md and split commits.
Diagrams drift from DDL again	Freeze schema v2 in Phase 1. Any later schema change updates migration + ER + relational diagram in the same PR (a PR checklist item).
Evaluator says "pgvector inside Postgres isn't a second paradigm"	Avoided by design: Qdrant is a separate engine with its own creation script and dashboard.
Embedding model slow on first call	Warm it up at server start. Cache the pipeline as a module singleton. The model downloads once and is cached afterwards.
Qdrant down or out of sync	SQL write never depends on Qdrant. `vector_synced_at` + reindex script + Vector Console status badge.
RLS blocks trigger side-effects	SECURITY DEFINER for cross-user writes (notifications, history) with a pinned `search_path`, covered by SQL tests.
Demo machine issues	Both laptops run the stack. A deterministic seed lets you reset in 30 seconds. Record a backup demo video.
Scope creep (fancy UI before DB marks)	Phases 2–3 (DB, 25+ marks) come before UI polish. React Bits stays on public pages only.

Next step

Phase 0 and Phase 1 can start now. Say the word and I'll scaffold the repo (Next.js 16, docker-compose with Postgres + pg_cron + Qdrant, the migration runner and schema v2 migrations 001–002) on a feature branch. You and Ashvin then review it and commit under your own names.